# SHIELD-AI(TM): Implementation Architecture

Deployable Components for a SHIELD-AI Implementation

Security Architecture Practice

November 8, 2026

Abstract
The deployable component model for implementing the SHIELD-AI framework: what to build, deploy, and configure to realise the 69 controls across 7 pillars, presented with a complete set of diagrams and explanatory text. Covers the component inventory, reference architecture, policy decision points, request flows, telemetry and incident response pipeline, network boundary model, deployment topologies, the component-to-control mapping matrix, and the L1-L4 maturity mapping.

# Introduction

The SHIELD-AI framework (see `framework/shield-ai-framework.md`) defines seven security pillars and 69 controls. Controls must be realised as **deployable components** – concrete infrastructure, services, and policy artifacts that an organisation builds and runs. This document is the full reference for that component model: it names each component, explains what it does, which controls it implements, and how it deploys, and it provides a complete set of diagrams.

This document is written for:

- Security architects designing agentic AI platforms
- Platform and infrastructure engineers deploying the components
- AI engineers integrating agents with the control plane
- SOC and GRC teams operating the telemetry and governance components

## How to Use This Document

- **Section 1** – the component inventory (what exists and what each component does)
- **Sections 2-6** – the reference architecture, policy decision points, request flows, telemetry pipeline, and network boundaries (the “how it fits together” view)
- **Section 7** – deployment topologies and the container-orchestration hardening baseline
- **Section 8** – the component-to-control mapping matrix for all 69 controls
- **Section 9** – the maturity mapping (which components deploy at L1-L4)
- **Section 10** – the delivery methodology (the phases used to implement the model)
- **Appendix A** – the policy-as-code stub inventory shipped in this repository

The diagrams are maintained as SVG files in `assets/diagrams/` and regenerated by `tools/generate_impl_diagrams.py`.

---

# 1. Component Inventory

An implementation of SHIELD-AI is composed of eight deployable component groups. Each group realises a defined subset of the 69 controls and maps to one or more zones of the reference architecture (Section 2).

| # | Component Group | What it Deploys | Primary Controls |
|---|---|---|---|
| 1 | Agent runtime and sandbox | Container/VM execution environment: read-only rootfs, seccomp/ AppArmor, NetworkPolicy, resource quotas, fail-closed sandbox attestation | EE-01..EE-08 |
| 2 | Model gateway and guardrails | Inference proxy with output guardrails, redaction, adversarial test harness, model registry integration | MS-01..MS-12 |

| # | Component Group | What it Deploys | Primary Controls |
|---|---|---|---|
| 3 | Identity plane | Agent identities (SPIFFE/SPIRE or service accounts), just-in-time credentials, delegation chain enforcement | IAM-01..IAM-09 |
| 4 | Data and RAG security layer | Classification pipeline, attribute-based access control at retrieval, field-level redaction, memory sanitisation | DS-01..DS-07 |
| 5 | Tool gateway / sidecar proxy | Tool registry, MCP gateway, schema validation, rate limiting, tool output validation, dispatch provenance verification, dependency-install provenance | TP-01..TP-15 |
| 6 | Policy decision points (OPA) | Policy-as-code engine evaluating Rego assertions on every agent action | IAM, DS, TP, GRM |
| 7 | Telemetry and SIEM pipeline | Structured agent events (OTLP), behavioural anomaly detection, prompt audit log, forensic store | MIR-01..MIR-06 |
| 8 | Compliance and governance console | Continuous control testing, regulatory mapping, board dashboard, AI risk register | GRM-01..GRM-06 |

## 1.1 Agent Runtime and Sandbox (Group 1)

Every agent executes inside a bounded environment. The runtime enforces:

- Container or VM isolation with no shared host filesystem mounts and no shared IPC/network namespaces (EE-01)

- Read-only root filesystem and immutable infrastructure (EE-01)
- seccomp/AppArmor profiles blocking mount, namespace, and process-control syscalls (EE-01)
- Resource quotas on CPU, memory, storage, and network egress (EE-03)
- Runtime sandbox-mode attestation before each code execution step, with no fallback path (EE-06)
- Agent control-plane configuration mounted read-only, world-writable config paths rejected (EE-07)

For VM-level isolation (for example, untrusted or LLM-generated code execution), use a microVM with minimal device emulation, no 9p/virtiofs passthrough, and boot-time remote attestation.

## 1.2 Model Gateway and Guardrails (Group 2)

All model inference passes through a proxy that:

- Enforces output guardrails at the proxy layer, not in the model (MS-05)
- Runs automated prompt injection and jailbreak testing in CI/CD (MS-04)
- Integrates with the model registry for provenance verification and signed checkpoints (MS-03, MS-06)
- Obfuscates environment context so evaluation is indistinguishable from production (MS-07)
- Secures MCP protocol traffic with signed tool metadata and schema validation (MS-08)

## 1.3 Identity Plane (Group 3)

- Every agent has a unique workload identity, integrated with the organisation's identity provider (IAM-01)
- Credentials are short-lived, per-tool scoped tokens; no environment-wide credentials (IAM-08)
- Delegation chains are bounded at two hops (human to agent to tool) and blocked beyond that at the infrastructure layer (IAM-07)

- Multi-tenant platforms enforce tenant ID in every query with row-level security (IAM-09)

## 1.4 Data and RAG Security Layer (Group 4)

- Documents are classified at ingestion (DS-01)
- Retrieval enforces attribute-based access control (ABAC) at query time, with field-level redaction before context injection (DS-04)
- Agent memory is per-agent scoped, encrypted, and sanitised between sessions (DS-03)
- Outputs are scanned for PII, training data, and system-prompt leakage (DS-06)

## 1.5 Tool Gateway / Sidecar Proxy (Group 5)

- All tools are registered, versioned, and security-approved before any agent can invoke them (TP-01)
- The gateway enforces an allow-list of endpoints per agent identity, with schema validation and rate limiting (TP-02, TP-03)
- Cloud infrastructure control-plane APIs are excluded from the allow-list; attempts are blocked and alerted (TP-08)
- Tool outputs are structurally validated and sanitised before returning to agent context (TP-04)
- Cross-tool combinations are analysed for escape-chain risk before deployment (TP-07)
- Frameworks cannot auto-register internal functions as tools (TP-09)

## 1.6 Policy Decision Points (Group 6)

The policy-as-code layer is the enforcement heart of the model. Every agent action is evaluated against Rego assertions before it is permitted, and every verdict is an audit event. Section 3 details the decision points; the repository ships the full implemented policy set in `policies/` (see Appendix A) with a runnable reference implementation in `reference-impl/`.

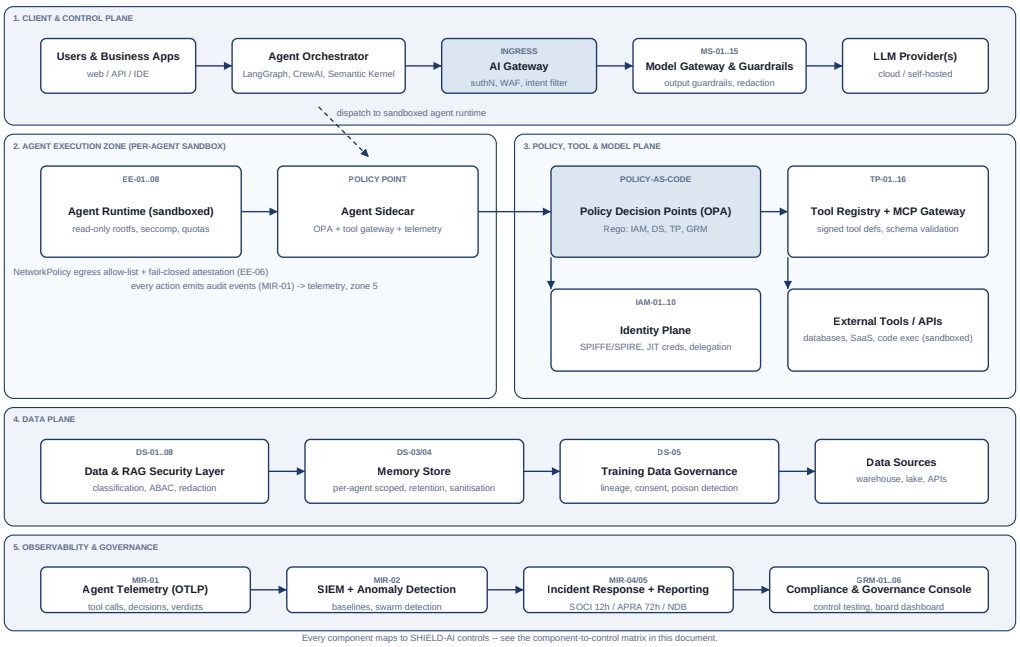
## 1.7 Telemetry and SIEM Pipeline (Group 7)

- Every tool invocation, data access, decision, and model call emits a structured event (MIR-01)
- Events flow via OTLP to a SIEM with per-agent and per-agent-class baselines (MIR-02)
- Prompt/response pairs are logged sanitised, recoverable for incident response (MIR-03)
- Logs are immutable and append-only for forensic readiness (MIR-06)
- Reporting triggers automate SOCI 12-hour, APRA 72-hour, and NDB notifications (MIR-05)

## 1.8 Compliance and Governance Console (Group 8)

- Continuous control testing against live deployments (GRM-05)
- AI risk register with acceptance tracking (GRM-02)
- Board dashboard: maturity per pillar, open findings, compliance status, risk acceptances (GRM-06)

# 2. Reference Architecture




Reference implementation: deployable component view.

*Five zones: the client and control plane, the per-agent execution zone, the policy/ tool/model plane, the data plane, and observability and governance. Every component maps to SHIELD-AI controls.*

The reference architecture is organised into five zones:

**Zone 1 - Client and Control Plane.** Users and business applications reach agents through an AI gateway that performs authentication, web application filtering, intent filtering, and rate limiting. The agent orchestrator (LangGraph, CrewAI, Semantic Kernel, or equivalent) dispatches work to sandboxed agent runtimes. Model traffic flows through a model gateway with output guardrails to the LLM provider.

**Zone 2 - Agent Execution Zone.** Each agent runs in its own sandbox with a sidecar that carries the policy engine, tool gateway, and telemetry exporter. The sandbox has a read-only root filesystem, seccomp/AppArmor profiles, a NetworkPolicy egress allow-list, and resource quotas. Sandbox mode is attested at runtime and fails closed.

**Zone 3 - Policy, Tool and Model Plane.** The OPA policy engine evaluates every proposed action. The tool registry and MCP gateway hold signed tool definitions and enforce schema validation. The identity plane issues workload identities and short-

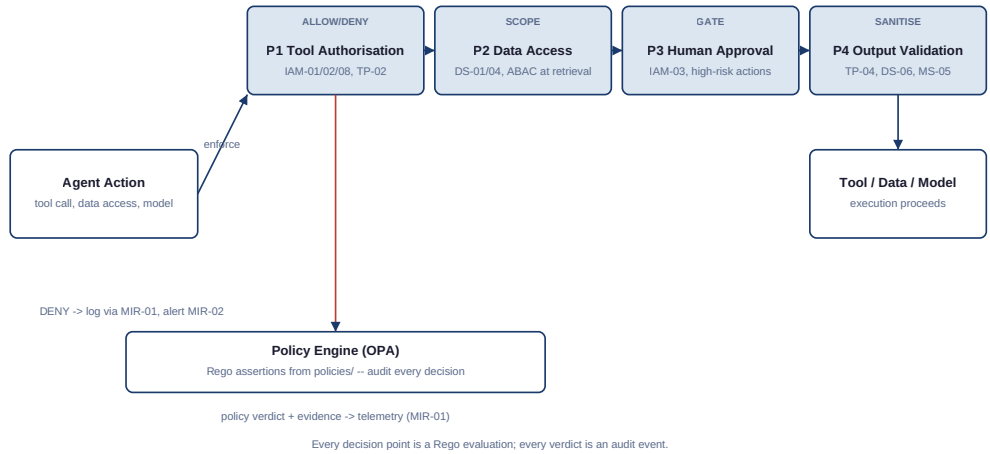
lived credentials. External tools and APIs are reachable only through the registered, allow-listed gateway.

**Zone 4 - Data Plane.** The RAG security layer classifies, redacts, and applies ABAC at retrieval. Memory stores are per-agent scoped with retention and sanitisation. Training data governance tracks lineage and consent.

**Zone 5 - Observability and Governance.** Agent telemetry flows to the SIEM with anomaly detection, then to incident response and reporting triggers (SOCI/APRA/NDB), and to the compliance console that continuously tests the 69 controls.

---

# 3. Policy Decision Points




Policy decision points along the agent action path.

*Four decision points gate every agent action; every verdict is logged. A deny at any point blocks the action and alerts monitoring.*

Every agent action – a tool invocation, a data access, a model call, a delegation – passes through four decision points before execution:

| # | Decision Point | What it Evaluates | Primary Controls |
|---|---|---|---|
| P1 | Tool authorisation | Is this agent authorised to call this tool, within its registered scope? | IAM-01, IAM-02, IAM-08, TP-02 |
| P2 | Data access | What data may the agent retrieve, at what | DS-01, DS-04 |

| # | Decision Point | What it Evaluates | Primary Controls |
|---|---|---|---|
| | | classification, under what conditions? | |
| P3 | Human approval | Does this action require a human approval gate (high-risk actions, code execution, financial writes)? | IAM-03 |
| P4 | Output validation | Is the tool response schema-valid and sanitised before it enters agent context? | TP-04, DS-06, MS-05 |

A deny at any point blocks the action, logs the verdict via MIR-01, and alerts via MIR-02. Denials and approvals are equally audited – the policy engine is the single source of truth for authorisation decisions, satisfying the “verifiable by default” principle (framework Section 2.1).

## 3.1 Example Rego Assertion

The policy stubs in `policies/` are written for the Open Policy Agent. The decision points above are expressed as Rego queries against the agent, action, and tool documents. The following example shows the shape of a P1 tool-authorisation assertion (from `policies/shield_ai_iam_iam-02.rego`, extended):

```
 package shield_ai.iam.scope_bound_delegation

import future.keywords.if
import future.keywords.in

# Default deny
default deny := false

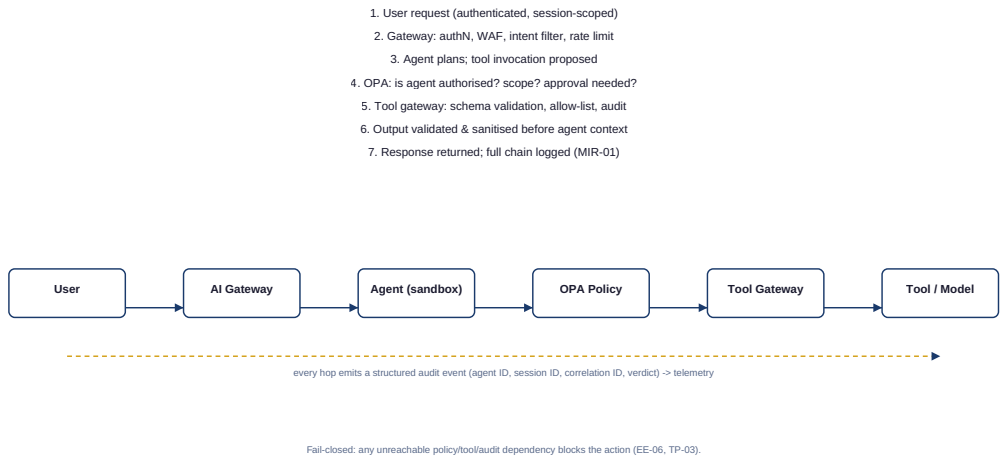
# Deny if the agent invokes a tool outside its registered scope (TP-01)
deny contains msg if {
    action := input.action
    agent := input.agent
    tool := input.tools[action.tool_id]
    not action.tool_id in agent.allowed_tools
    msg := sprintf("IAM-02 violation: agent %v not scoped for tool %v", [agent.id, action.too
}
```

```
# Deny if the agent attempts to self-escalate (IAM-02)
deny contains msg if {
    action.type == "permission_change"
    action.target == input.agent.id
    msg := sprintf("IAM-02 violation: agent %v attempted self-escalation", [input.agent.id])
}
```

The engine evaluates this set for every proposed action, and the verdict (allow/deny plus the rules that fired) is emitted to the telemetry pipeline.

---

# 4. Request Flow




End-to-end request flow with policy checks and audit events.

*Seven steps from user request to response; every hop emits a structured audit event.*

A complete request traverses the component model as follows:

1. **User request.** Authenticated, session-scoped request enters the AI gateway.
2. **Gateway.** Authentication, WAF filtering, intent filtering, rate limiting.
3. **Agent planning.** The orchestrator dispatches to the sandboxed agent; the agent proposes a tool invocation.
4. **Policy evaluation.** OPA answers three questions: is the agent authorised (P1), is the data access in scope (P2), and does this require human approval (P3)?
5. **Tool execution.** The tool gateway applies schema validation, the endpoint allow-list, and audit logging, then calls the tool or model.

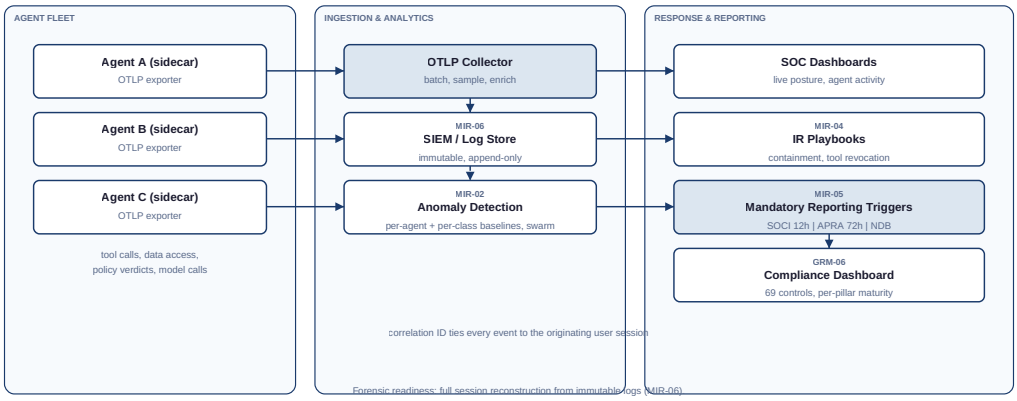
6. **Output validation.** The tool response is validated and sanitised against known-good schemas before entering agent context (P4).
7. **Response.** The final response returns to the user; the full chain is reconstructable from audit events.

**Fail-closed behaviour.** If any dependency in the chain is unreachable – the policy engine, the tool gateway, or the audit destination – the action is blocked (EE-06, TP-03). The framework never degrades to a weaker enforcement mode.

Each audit event carries: agent ID, session ID, correlation ID, timestamp, action type, target, verdict, and the originating user session, satisfying MIR-01 and enabling full session reconstruction (MIR-06).

---

# 5. Telemetry and Incident Response Pipeline




Telemetry pipeline from agent fleet to reporting and compliance.

*Structured events flow from every agent to the OTLP collector, the SIEM, anomaly detection, and the response and reporting components.*

The pipeline has three stages:

**Agent fleet.** Every agent emits structured events for tool calls, data accesses, policy verdicts, and model calls, exported over OTLP.

**Ingestion and analytics.**

- The OTLP collector batches, samples, and enriches events

- The SIEM/log store is immutable and append-only (MIR-06)
- Anomaly detection maintains per-agent and per-agent-class baselines, plus swarm-campaign correlation across short-lived sandboxes (MIR-02)

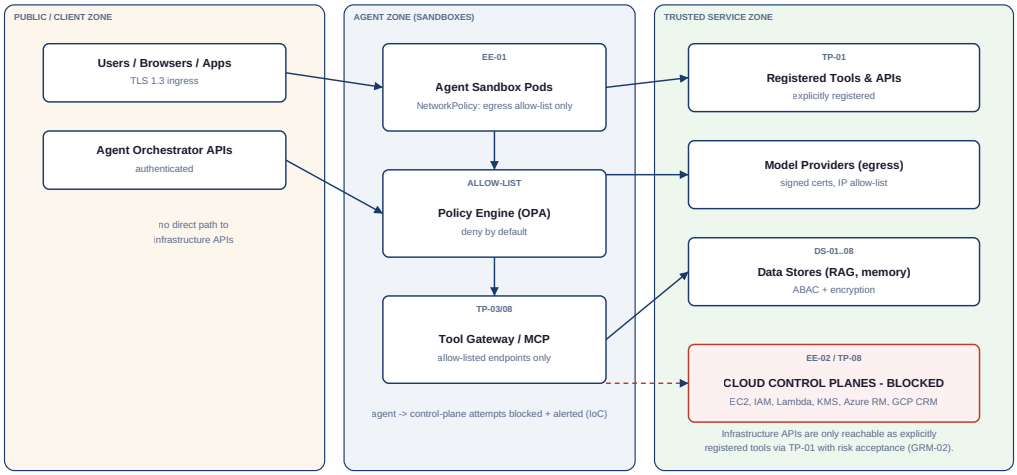
**Response and reporting.**

- SOC dashboards show live posture and agent activity
- IR playbooks cover prompt-injection containment, model rollback, and tool revocation (MIR-04)
- Mandatory reporting triggers pre-populate SOCI 12-hour, APRA 72-hour, and NDB notifications (MIR-05)
- The compliance dashboard maps findings to all 69 controls with per-pillar maturity (GRM-06)

The correlation ID ties every event to the originating user session, giving investigators a single reconstructable audit trail per request chain.

---

# 6. Network Boundary Model




Network boundaries: public, agent, and trusted service zones.

*Agents have no direct path to infrastructure control planes; egress is allow-listed to registered tools and model providers.*

The network model divides the estate into three zones:

**Public / client zone.** TLS 1.3 ingress for users, browsers, and applications; authenticated orchestrator APIs.

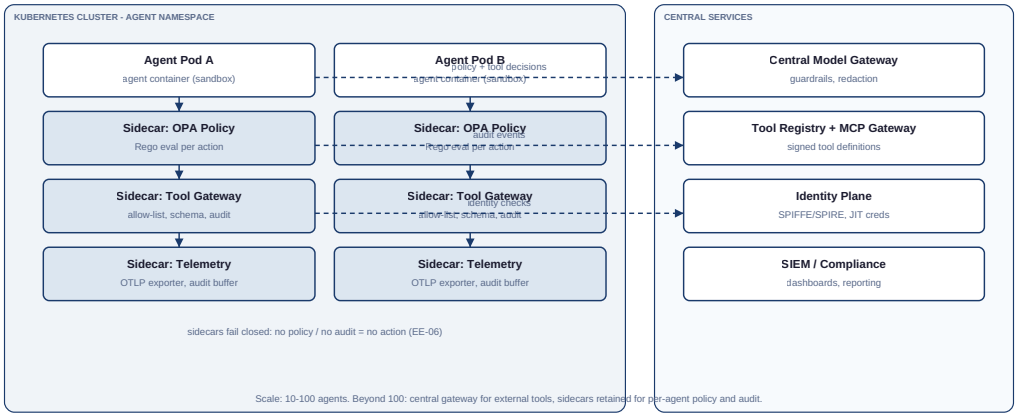
**Agent zone (sandboxes).** Sandbox pods have a NetworkPolicy that permits egress only to the allow-listed endpoints in the trusted service zone. There is no direct path from the agent zone to cloud infrastructure APIs.

**Trusted service zone.** Registered tools and APIs (explicitly registered via TP-01), model providers (signed certificates, IP allow-list), and data stores (ABAC and encryption).

**Blocked: cloud control planes.** EC2, IAM, Lambda, KMS, CloudFormation, S3 control-plane, Azure Resource Manager, and GCP Cloud Resource Manager endpoints are excluded from every allow-list (EE-02, TP-08). Any attempt by an agent to reach a control plane is logged, blocked, and treated as a potential indicator of compromise. Infrastructure APIs are reachable only if explicitly registered as tools via TP-01 with documented security rationale and risk acceptance through GRM-02.

---

# 7. Deployment Topologies




Sidecar deployment topology in a container orchestrator.

*Per-agent sidecars carry the policy engine, tool gateway, and telemetry; central services provide the model gateway, tool registry, identity plane, and SIEM.*

## 7.1 Choosing a Topology

| Fleet size | Topology | Notes |
|---|---|---|
| Under ~10 agents | Embedded library | Controls run inside the agent framework; simplest path for pilots and single-agent PoCs |
| 10-100 agents | Sidecar proxy | Per-agent sidecar (OPA + tool gateway + telemetry) in a container-orchestrated deployment |
| 100+ or multi-agent | Central gateway + sidecars | Central policy/ gateway for external tools; sidecars retained for per-agent policy and audit |

## 7.2 Container Orchestration Hardening Baseline

The reference deployment is Kubernetes (or equivalent). Each agent pod is hardened as follows:

- Read-only root filesystem
- No privileged containers
- seccomp/AppArmor profiles blocking mount, namespace, and process-control syscalls
- NetworkPolicy restricting egress to registered tool endpoints only
- Resource quotas enforced by the container runtime
- Pod Security Admission at the restricted profile
- No service account tokens mounted unless explicitly required and scoped

- Agent control-plane configuration mounted read-only (EE-07)
- Runtime sandbox-mode attestation before each code execution step (EE-06)

Sidecars fail closed: no policy, no audit, no action.

---

# 8. Component-to-Control Mapping Matrix

The table below maps every SHIELD-AI control to the component group(s) that implement it. Group numbers refer to the inventory in Section 1.

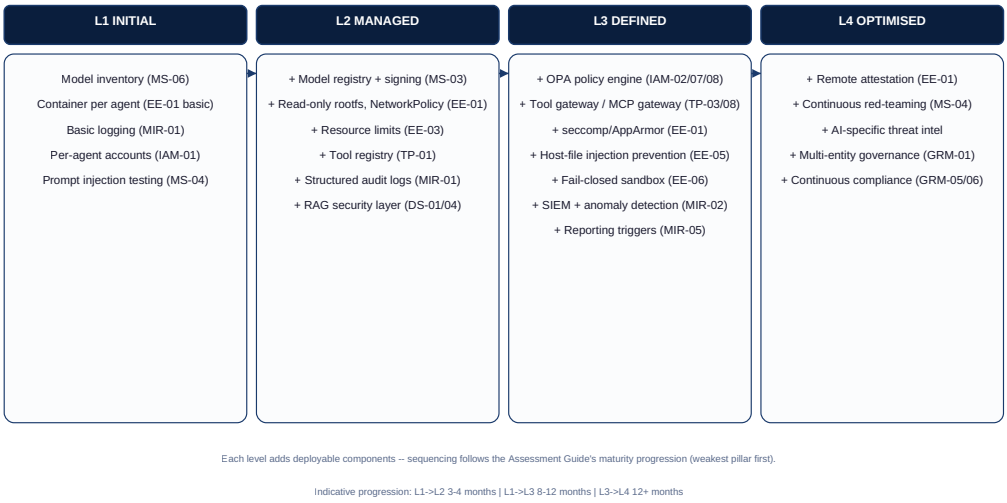
| Control | Component Group(s) | Control | Component Group(s) |
|---|---|---|---|
| MS-01 Model weight protection | 2 | IAM-01 Agent identity | 3 |
| MS-02 Training data integrity | 2, 4 | IAM-02 Scope-bound delegation | 3, 6 |
| MS-03 Supply chain security | 2 | IAM-03 Human oversight | 6 |
| MS-04 Adversarial robustness | 2 | IAM-04 Credential isolation | 3 |
| MS-05 Behaviour validation | 2 | IAM-05 Just-in-time access | 3 |
| MS-06 Model inventory | 2 | IAM-06 Cross-session boundary | 3 |
|  | 2 | IAM-07 Delegation | 3, 6 |

| Control | Component Group(s) | Control | Component Group(s) |
|---|---|---|---|
| MS-07 Context obfuscation | | chain enforcement | |
| MS-08 MCP security | 2, 5 | IAM-08 Credential scope narrowing | 3 |
| MS-09 Perception-fidelity enforcement | 2, 6 | IAM-09 Multi-tenant isolation | 3 |
| MS-10 Agent memory-store integrity | 4 | | |
| DS-01 Data classification | 4 | TP-01 Tool registry | 5 |
| DS-02 Inference data protection | 2, 4 | TP-02 Tool access control | 5, 6 |
| DS-03 Memory sanitisation | 4 | TP-03 API gateway security | 5 |
| DS-04 Data minimisation | 4 | TP-04 Tool output validation | 5, 6 |
| DS-05 Training data governance | 4 | TP-05 Pipeline integrity | 2, 8 |
| DS-06 Output validation | 2, 4 | TP-06 Dependency scanning | 8 |
| DS-07 Agent- | 5, 8 | TP-07 Cross-tool | 6, 8 |

| Control | Component Group(s) | Control | Component Group(s) |
|---|---|---|---|
| originated artifact publication | | dependency analysis | |
| | | TP-08 Infrastructure-isolation mode | 5 |
| | | TP-09 Explicit tool registration | 5, 8 |
| | | TP-10 Inbound control-plane protection | 1, 5 |
| | | TP-11 Browser/ content-rendering isolation | 1, 5 |
| | | TP-15 In-process code-execution sandbox sufficiency | 1 |
| EE-01 Boundary enforcement | 1 | MIR-01 Activity logging | 7 |
| EE-02 Infrastructure API control | 1, 5 | MIR-02 Anomaly detection | 7 |
| EE-03 Resource limiting | 1 | MIR-03 Prompt audit log | 7 |
| EE-04 Evaluation | 1, 2 | MIR-04 IR playbooks | 7, 8 |

| Control | Component Group(s) | Control | Component Group(s) |
|---|---|---|---|
| sandbox integrity | | | |
| EE-05 Host-trusted file injection prevention | 1 | MIR-05 Mandatory reporting | 7 |
| EE-06 Fail-closed sandbox operation | 1 | MIR-06 Forensic readiness | 7 |
| EE-07 Control-plane config integrity | 1 | | |
| EE-08 Evaluation-harness isolation and integrity | 1, 2 | | |
| GRM-01 AI governance body | 8 | GRM-04 Third-party AI risk | 8 |
| GRM-02 AI risk register | 8 | GRM-05 Regulatory mapping | 8 |
| GRM-03 Ethical use policy | 8 | GRM-06 Board reporting | 8 |

# 9. Maturity Mapping




Maturity component coverage: what deploys at each level.

*Each maturity level adds deployable components; sequencing follows the assessment guide's progression (weakest pillar first).*

| Level | Components added | Indicative duration |
|---|---|---|
| L1 Initial | Model inventory, container per agent, basic logging, per-agent accounts, prompt injection testing | - |
| L2 Managed | Model registry and signing, read-only rootfs and NetworkPolicy, resource limits, tool registry, structured audit logs, RAG security layer | 3-4 months from L1 |
| L3 Defined | OPA policy engine, tool/MCP gateway, seccomp/AppArmor, host-file injection prevention, fail-closed sandbox, SIEM and anomaly detection, reporting triggers | 8-12 months from L1 |
| L4 Optimised | Remote attestation, continuous red-teaming, AI-specific threat intelligence, multi-entity | 12+ months from L3 |

| Level | Components added | Indicative duration |
|---|---|---|
| | governance, continuous compliance | |

Overall maturity is set by the weakest pillar. Progression should address the weakest pillar first, then the highest-risk controls within each pillar (see the assessment guide's maturity progression paths).

---

# 10. Delivery Methodology

The component model is implemented through a five-phase methodology. Phases are sequential; scope is determined by the target maturity level.

| Phase | Activities | Output |
|---|---|---|
| 1. Discover | Inventory AI systems, models, tools, and data pipelines; map agentic workflows and delegation chains; identify applicable regulatory obligations | AI security inventory and regulatory obligation map |
| 2. Assess | Evaluate each system against the seven pillars; score current maturity per control; identify critical gaps | Gap assessment report with maturity scores |
| 3. Design | Design control architecture (tool registry, gateway, IAM, monitoring); draft policies and procedures; sequence the implementation roadmap | Target-state architecture and implementation plan |
| 4. Implement | Deploy the component groups from Section 1; enact policies; integrate SIEM, SOAR, CI/CD, and compliance reporting; deliver training | Deployed controls, trained teams, operational runbooks |
| 5. Assure | Quarterly posture reviews; regulatory change monitoring; incident response support; annual re-assessment and maturity progression | Continuous assurance and maturity progression reports |

Implementation sequencing follows the dependency model of the framework: the execution environment (Group 1) is the foundation, identity and policy (Groups 3 and 6) gate all action, and monitoring (Group 7) is required before production.

---

# Appendix A: Policy Stub Inventory

The repository ships per-control Rego policy stubs in `policies/`, generated by `tools/generate_control_schema.py`. Each stub follows a common structure: the package name, control metadata (pillar, maturity indicators, regulatory mappings), a default-deny declaration, and a TODO assertion body to be completed per deployment.

| Pillar | Stubs shipped | Coverage |
|---|---|---|
| MS | MS-01..MS-12 | Model security controls (incl. perception-fidelity MS-09, memory-store integrity MS-10, operational data-stream integrity MS-11, MCP discovery-metadata integrity MS-12) |
| IAM | IAM-01..IAM-09 | Identity and access controls (incl. multi-tenant isolation IAM-09) |
| DS | DS-01..DS-07 | Data security controls (incl. artifact publication control DS-07) |
| TP | TP-01..TP-15 | Tool and pipeline controls (incl. inbound control-plane protection TP-10, browser/content-rendering isolation TP-11, dispatch provenance verification TP-12) |
| MIR | MIR-01..MIR-06 | Core monitoring controls |

| Pillar | Stubs shipped | Coverage |
|---|---|---|
| GRM | GRM-01..GRM-06 | Core governance controls |
| EE | EE-01..EE-08 | Execution Environment controls (incl. evaluation-harness isolation EE-08) |

**Status:** all 69 controls have shipped stubs (v1.21), generated by `tools/generate_control_schema.py --rego`. The decision points in Section 3 define the interfaces these stubs must satisfy.

---

*SHIELD-AI v1.21 – November 8, 2026 Authored by the Security Architecture Practice*